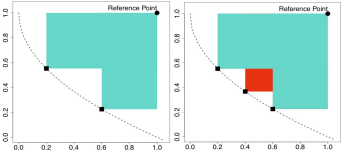
Optimization in Machine Learning

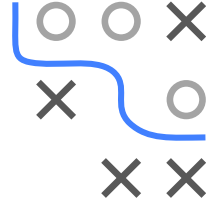
Multi-criteria Optimization
Bayesian Optimization



Learning goals

- Single-criteria recap
- Multi-criteria introduction
- Scalarization vs. HV-based methods

RECAP: BAYESIAN OPTIMIZATION

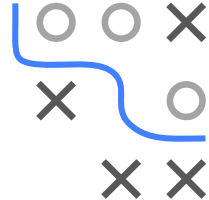


Advantages of BO

- Sample efficient
- Can handle noise
- Native incorporation of priors
- Does not require gradients
- Theoretical guarantees

We will now extend BO to multiple cost functions.

RECAP: BAYESIAN OPTIMIZATION



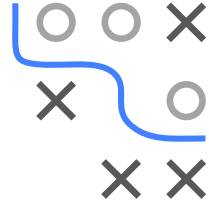
Algorithm Bayesian optimization loop

Require: Search space $\mathcal{S}$, cost function f, acquisition function u, predictive model $\hat{f}$, maximal number of function evaluations T

Ensure: Best configuration $\hat{\boldsymbol{x}}$ (based on $\mathcal{D}$ or $\hat{f}$)

```
Initialize data D^(0) with initial observations
for t ← 1 to T do
    Fit predictive model f̂^(t) on D^(t−1)
    Select next query point:
    x^(t) ∈ arg max_{x∈S} u(x; D^(t−1), f̂^(t))
    Query f(x^(t))
    Update data:
    D^(t) ← D^(t−1) ∪ {⟨x^(t), f(x^(t))⟩}
end for
```

MULTI-CRITERIA BAYESIAN OPTIMIZATION



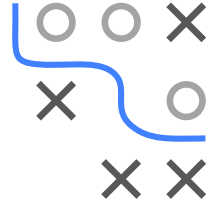
Goal: Extend Bayesian optimization to multiple cost functions

$$\min_{\boldsymbol{x}\in\mathcal{S}} f(\boldsymbol{x}) \quad \Leftrightarrow \quad \min_{\boldsymbol{x}\in\mathcal{S}} \big(f_1(\boldsymbol{x}), f_2(\boldsymbol{x}), \ldots, f_m(\boldsymbol{x})\big).$$

There are two basic approaches:

1. Simplify the problem by scalarizing the cost functions, or
2. define acquisition functions for multiple cost functions.

SCALARIZATION

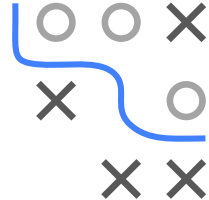


Idea: Aggregate all cost functions

$$\min_{\boldsymbol{x} \in \mathcal{S}} \sum_{i=1}^{m} w_i f_i(\boldsymbol{x}) \qquad \text{with} \quad w_i \geq 0$$

- **Obvious problem:** How to choose $w_1, \ldots, w_m$?
  - Expert knowledge?
  - Systematic variation?
  - Random variation?
- If expert knowledge is not available a priori, we need to ensure that different trade-offs between cost functions are explored.
- Simplifies multi-criteria optimization problem to single-objective $\longrightarrow$ standard Bayesian optimization can be used without adapting the general algorithm.

SCALARIZATION: PAREGO ▸ Knowles 2006



Scalarize the cost functions using the augmented Chebyshev norm / achievement function

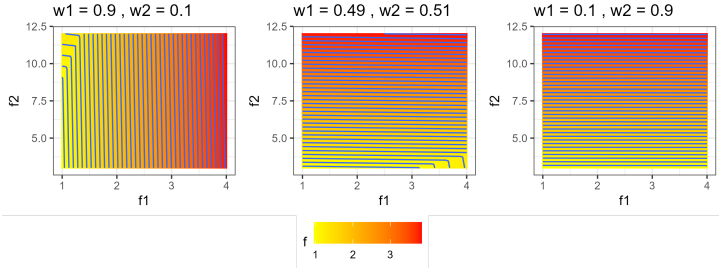
$$f = \max_{i=1,\dots,m} \big(w_i f_i(\boldsymbol{x})\big) + \rho \sum_{i=1}^{m} w_i f_i(\boldsymbol{x}),$$

- The weights $w \in W$ are drawn from

$$W = \Big\{ w = (w_1, \dots, w_m) \,\Big|\, \sum_{i=1}^{m} w_i = 1,\ w_i = \tfrac{l}{s},\ l \in \{0, \dots, s\} \Big\},$$

  with $|W| = \binom{s+m-1}{m-1}$.
- New weights are drawn in every BO iteration.
- ρ is a small parameter (e.g. 0.05).
- s controls how many distinct weight vectors exist.

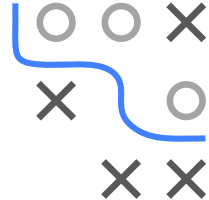
WHY THE CHEBYSHEV NORM?



$$f = \max_{i=1,\ldots,m}\big(w_i f_i(\boldsymbol{x})\big) + \rho \sum_{i=1}^{m} w_i f_i(\boldsymbol{x}),$$

- The norm consists of two components:
  - $\max_i\big(w_i f_i(\boldsymbol{x})\big)$ considers only the maximum weighted cost.
  - $\sum_i w_i f_i(\boldsymbol{x})$ is the weighted sum of all costs.
- ρ controls the trade-off between these parts.
- By randomizing the weights w in each iteration (and keeping ρ small), extreme points in single cost functions can be explored.
- One can prove that every solution of this scalarized problem is Pareto-optimal.

PAREGO ALGORITHM



Algorithm ParEGO loop

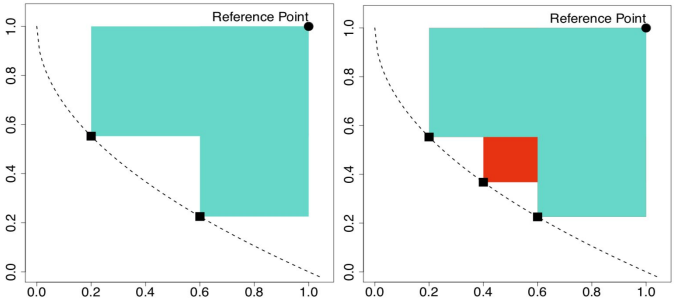
Require: Search space $\mathcal{S}$, cost function f, acquisition function u, predictive model $\hat{f}$, maximal number of function evaluations T, ρ, l, s

Ensure: Best configuration $\hat{\boldsymbol{x}}$ (according to $\mathcal{D}$ or $\hat{f}$)

```
Initialize data D^(0) with initial observations
for t = 1 to T do
    Sample w ∈ W = {(w_1, ..., w_m) | Σ w_i = 1, w_i = l/s, l ∈ {0, ..., s}}
    Compute scalarization: c^(t) = max_i (w_i f_i(x)) + ρ Σ_i w_i f_i(x)
    Fit predictive model f̂^(t) on D^(t−1)
    Select next query point: x^(t) ∈ arg max_{x∈S} u(x; D^(t−1), f̂^(t))
    Query f(x^(t))
    Update data: D^(t) ← D^(t−1) ∪ {⟨x^(t), f(x^(t))⟩}
end for
```

HV-BASED ACQUISITION FUNCTIONS

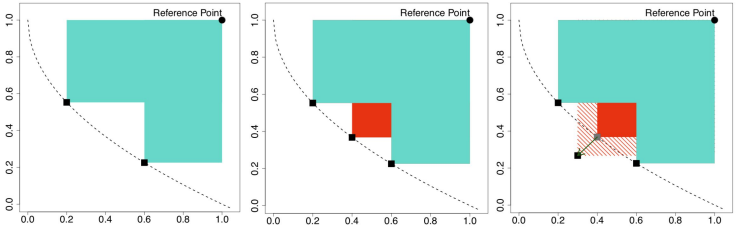
Idea: Define acquisition function that directly models the contribution to the dominated hypervolume (HV): $\max(0,\ S(\mathcal{P} \cup \boldsymbol{x}, R)\ -\ S(\mathcal{P}, R))$



- Fit m single-objective surrogate models $\hat{f}_1, \ldots, \hat{f}_m$.
- Acquisition function uses all surrogate models simultaneously.
- Still a single-objective optimization problem (of acquisition itself).

S-METRIC SELECTION-BASED EGO

Using the Lower Confidence Bound $u_{\mathrm{LCB},1}(\boldsymbol{x}), \ldots, u_{\mathrm{LCB},m}(\boldsymbol{x})$, one can form an *optimistic* estimate of the hypervolume contribution:



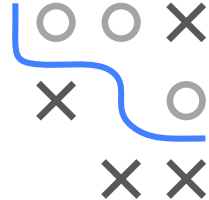
Problem: HV-contribution can be zero across large regions where the lower confidence bound is already dominated.

- Large zero-plateaus make optimization of the acquisition harder.
- SMS-EGO adds an adaptive penalty in such dominated regions.

This method is referred to as ▸ Ponweiser et al. 2008.

FURTHER HV ACQUISITION FUNCTIONS

Expected Hypervolume Improvement (EHI) ▸ Yang et al. 2019:



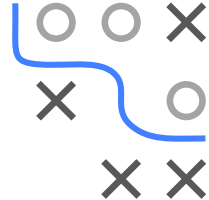
$$u_{EI,\mathcal{H}}(\boldsymbol{x}) = \int_{-\infty}^{\infty} p(f \mid \boldsymbol{x}) \times \left[S(\mathcal{P} \cup \boldsymbol{x}, R) \; - \; S(\mathcal{P}, R) \right] \, df$$

- Direct extension of u_{EI} to the hypervolume criterion.
- $p(f \mid \boldsymbol{x})$ is the joint predictive density of all surrogate models at $\boldsymbol{x}$.
- With independent GPs per objective, this factorizes over m univariate normals.
- Exact integral feasible for $m \leq 3$, else simulation-based.

Further HV-based acquisitions:

- **Stepwise Uncertainty Reduction (SUR)** w.r.t. the probability of improvement.
- **Expected Maximin Improvement (EMI)** w.r.t. the ϵ-indicator.

HYPERVOLUME-BASED BO ALGORITHM



Algorithm Hypervolume-based Bayesian optimization loop

Require: Search space $\mathcal{S}$, cost function f, acquisition function u, predictive model(s) $\hat{f}_1, \ldots, \hat{f}_m$, maximal number of function evaluations T

Ensure: Best configuration $\hat{\boldsymbol{x}}$

```
Initialize data D^(0) with initial observations
for t = 1 to T do
    Fit predictive models f̂_1^(t), ..., f̂_m^(t) on D^(t−1)
    Select next query point:
    x^(t) ∈ arg max_{x∈S} u(x; D^(t−1), f̂_1^(t), ..., f̂_m^(t))
    Query f(x^(t))
    Update data: D^(t) ← D^(t−1) ∪ {⟨x^(t), f(x^(t))⟩}
end for
```